

Three vertical lines of varying heights on the left side of the slide.

UT1

Introducción a la programación multiproceso

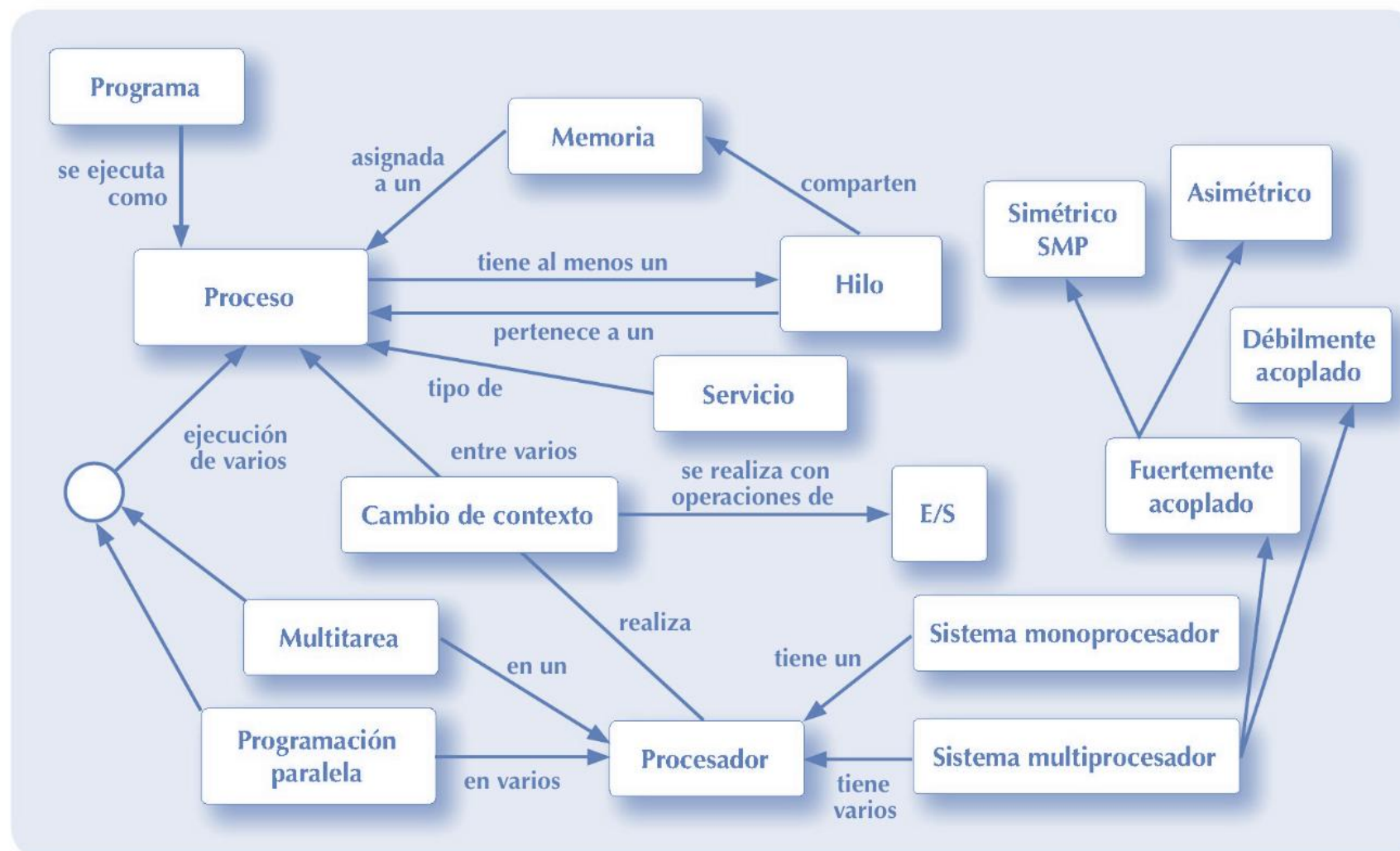
Programación de servicios y procesos

2º Desarrollo de Aplicaciones Multiplataforma

¿Qué vamos a trabajar?

- Comprender los conceptos básicos del funcionamiento de los sistemas en lo relativo a la ejecución de diferentes programas.
- Comprender el concepto de concurrencia y cómo el sistema puede proporcionar multiprogramación al usuario.
- Entender las políticas de planificación del sistema para proporcionar multiprogramación y multitarea.
- Familiarizarse con la programación de procesos, entendiendo sus principios y formas de aplicación.
- Resolver problemas concretos utilizando las clases de Java que permiten crear dinámicamente, y ejecutar, de manera concurrente, nuevos procesos.

¿Qué vamos a trabajar?



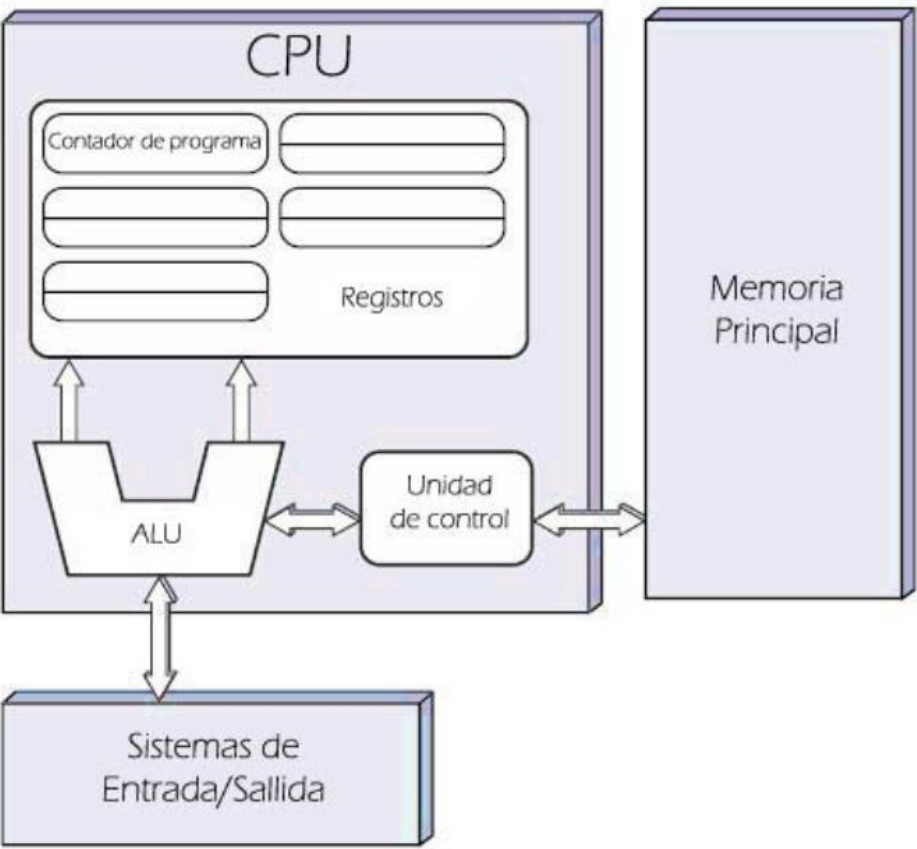
Programas y procesos

- **Programa:** toda la información almacenada en disco de una aplicación que resuelve una necesidad concreta.

Programa = instrucciones + datos

- **Proceso:** una instancia de un programa en ejecución. Incluye además que el código y los datos, todo lo necesario para la ejecución de dicho programa. Es una entidad dinámica. Su contexto está formado por:
 - Contador de programa, indica por dónde se está ejecutando.
 - Espacio en memoria, que está usando el proceso.
 - Estado del procesador, es el valor de los registros del procesador sobre los cuales se está ejecutando.
- Un proceso es independiente de otros procesos.
 - Puede haber más instancias del mismo programa ejecutándose en un determinado momento
- Los estados en los que puede estar son:
 - En ejecución: está dentro del microprocesador.
 - Pausado/detenido/en espera: el proceso tiene que seguir en ejecución pero en ese momento el S.O tomó la decisión de dejar paso a otro.
 - Interrumpido: el proceso tiene que seguir en ejecución pero el usuario ha decidido interrumpir la ejecución.
 - Existen otros estados pero ya son muy dependientes del sistema operativo concreto.

Programas y procesos



Estructura de un procesado: máquina Von Neumann

Para ver los procesos en Linux

- Comando *ps tree*

Para ver los procesos en Windows.

<https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer>

Procesos	Hilos
Constan de uno o más hilos	Un hilo siempre existe dentro de un proceso
Son independientes unos de otros	Comparten los recursos del proceso de forma directa
Son gestionados por el SO	Son gestionados por el proceso
Se comunican a través del SO	La comunicación la controla el proceso

Ejemplo código ensamblador

```
L0:  MOV    R1, #a    ;
      MOV    R2, #b    ;
L1:  LD     R3, (R1)   ;
      CMP    R3, #0    ;
      BNE    L3        ;
L2:  MOV    R4, #1     ;
      JMP    L4        ;
L3:  MOV    R4, #0     ;
L4:  ST     (R2), R4   ;
      JMP    L1        ;
```

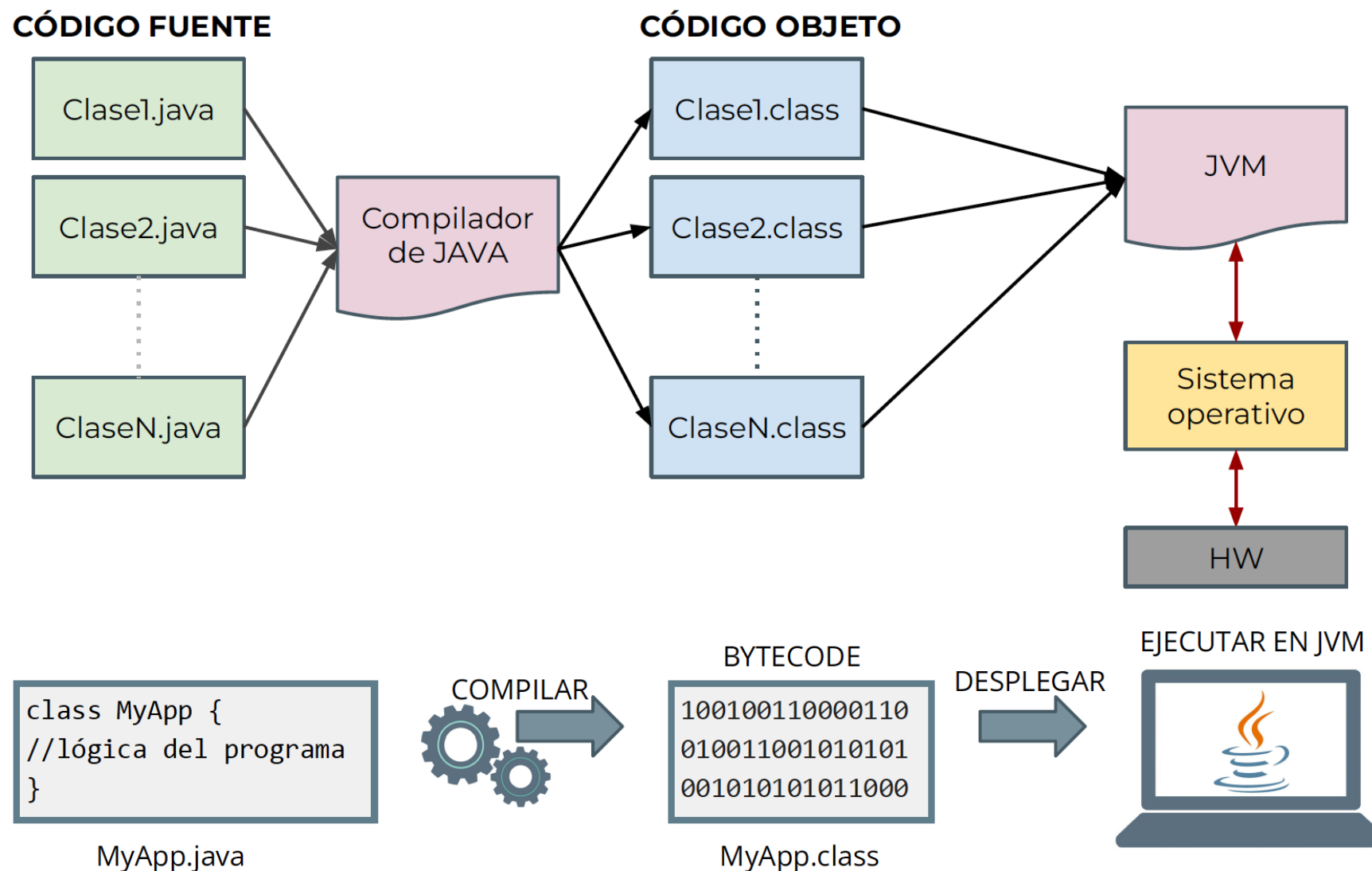
Ejemplo código binario

```
01010100 01101000 01101001 01110011
00100000 01101001 01110011 00100000
01110100 01101000 01100101 00100000
01110100 01110101 01110100 01101111
01110010 01101001 01100001 01101100
00100000 01110100 01101111 00100000
01101100 01100101 01100001 01110010
01101110 00100000 01100010 01101001
01101110 01100001 01110010 01111001
00101110 00100000 01001001 00100000
01101000 01101111 01110000 01100101
00100000 01111001 01101111 01110101
00100000 01100101 01101110 01101010
01101111 01111001 00100000 01101001
01110100 00100001
```

Ejecutable

- Un fichero Ejecutable contiene la información para crear un proceso a partir de la información de un programa.
 - .exe en Windows
 - Se pueden utilizar «desensambladores» para averiguar la secuencia de instrucciones que hay en un EXE
- Java no genera .exe sino .class o .jar
 - ¿por qué los .class no son ejecutables?
 - Son archivos que el intérprete de JAVA (el archivo java.exe) leerá y ejecutará.
 - El intérprete toma el programa y lo traduce a instrucciones del microprocesador en el que estemos, que puede ser x86 o un x64 o lo que sea. Ese proceso se hace «al instante» o JIT (Just-In-Time).
 - Java puede averiguar en qué microprocesador se está ejecutando y así generar el código más óptimo posible

Compilación y ejecución en Java



Compilación y ejecución

- Un fichero Ejecutable contiene la información para crear un proceso a partir de la información de un programa.
 - .exe en Windows
 - Se pueden utilizar «desensambladores» para averiguar la secuencia de instrucciones que hay en un EXE
- Java no genera .exe sino .class o .jar
 - ¿por qué los .class no son ejecutables?
 - Son archivos que el intérprete de JAVA (el archivo java.exe) leerá y ejecutará.
 - El intérprete toma el programa y lo traduce a instrucciones del microprocesador en el que estemos, que puede ser x86 o un x64 o lo que sea. Ese proceso se hace «al instante» o JIT (Just-In-Time).
 - Java puede averiguar en qué microprocesador se está ejecutando y así generar el código más óptimo posible

Multitarea (*multitasking*)

• ¿Qué es la multitarea?

- Ejecución simultánea de más de un proceso a lo largo de un intervalo regular de tiempo en el procesador.
- El sistema operativo juega un papel fundamental.
- Antes de empezar un nuevo proceso, se guarda el estado de ejecución del actual para poder retomarlo después en el mismo punto en el que se interrumpió.
- Generalmente habrá muchos más procesos que procesadores, por lo que también en los sistemas multiprocesador hay multitarea.
- Hay que tener en cuenta, además del tiempo de ejecución del proceso, también el tiempo de cambio de contexto, así como el del SO para la planificación.

- Ejecución de procesos sin multitarea

CPU	A	A	A	A	A	B	B	B	B	C	C	C	C	C	C
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- Ejecución de procesos con multitarea (monoprocesador)

A															
B															
C															
CPU	A	B	C	A	B	C	A	B	C	A	B	C	A	C	C
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- Ejecución de procesos en un sistema multiprocesador

CPU 1	A	A	A	A	A										
CPU 2	B	B	B	B											
CPU 3	C	C	C	C	C	C									
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

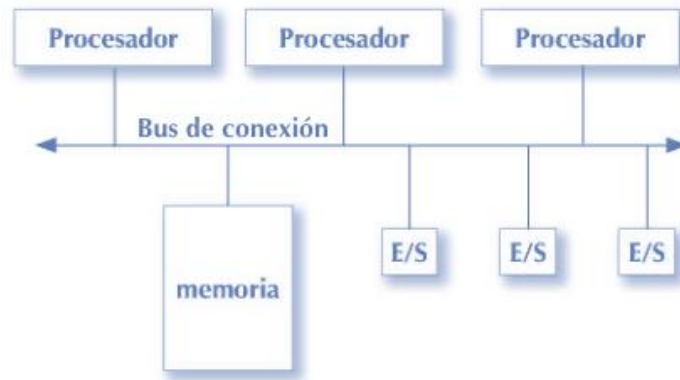
Programación concurrente

- ¿Qué es la **programación concurrente**?
 - **Procesos concurrentes**: se ejecutan de manera simultánea durante un intervalo de tiempo (de forma real o simulada).
 - **Programación concurrente**: técnicas que permiten desarrollar programas que utilizan varios procesos concurrentes que funcionan de forma conjunta y coordinada, para realizar una tarea.
 - **IMPORTANTE** la comunicación y la sincronización entre sí. Las técnicas variaran si es monoprocesador o multiprocesador.
- En **sistemas monoprocesadores** (un único procesador).
 - La técnica de cambio de contexto permite una multitarea efectiva y un mejor aprovechamiento del procesador, que se asigna a otro proceso durante el tiempo en que un proceso está inactivo, o a la espera de que se termine de realizar una operación E/S.
- En **sistemas multiprocesadores** (más de un procesador).
 - Se pueden clasificar en fuertemente acoplados o débilmente acoplados.
- ¿Qué es la **programación paralela**? ¿Y la **programación distribuida**?
 - **Programación paralela**. Técnicas que permiten desarrollar programas que utilizan varios procesos concurrentes que pueden ejecutarse en distintos procesadores.
 - **Programación distribuida**. Permite distribuir la carga computacional entre varios ordenadores conectados en red.

Programación concurrente

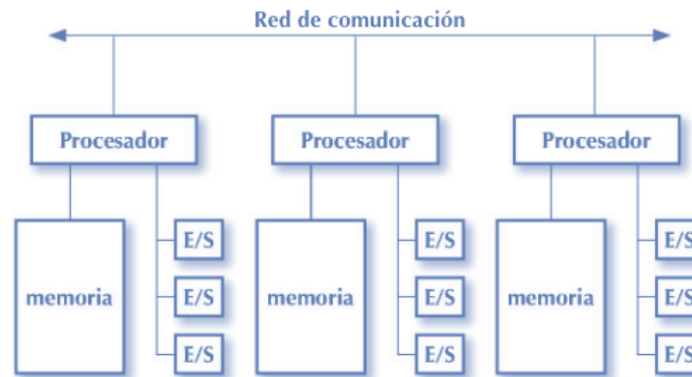
Sistemas multiprocesadores.

- Sistemas fuertemente acoplados
 - Memoria compartida para todos los procesadores, a la que todos acceden por un bus de conexión.

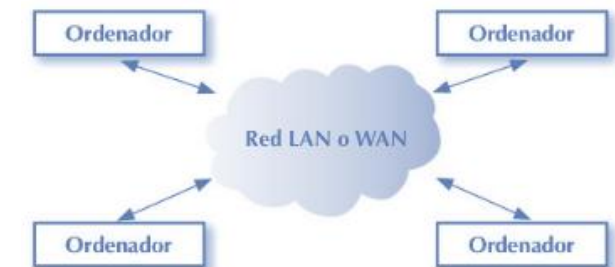


Pueden ser simétricos o asimétricos, con un modelo maestro-esclavo

- Sistemas débilmente acoplados
 - No hay memoria compartida, sino que cada procesador tiene su propia memoria y su propio sistema de E/S.



- Sistemas distribuidos
 - Los procesadores están en ordenadores independientes que se comunican por una LAN o WAN o Internet.
 - Son heterogéneos en cuanto al HW e independientes entre los sistemas.
 - Son altamente escalables y configurables, pero complejos de mantener



Kernel del sistema operativo

- ¿Qué es el *kernel*?
 - Es la parte central del sistema operativo; pequeña y muy optimizada, da respuesta a multitud de eventos mediante un mecanismo de gestión de interrupciones.
 - Una interrupción consiste en suspender temporalmente la ejecución de un proceso para que se pueda ejecutar una parte de código del *kernel* que está definida para tratar esa interrupción específica.
 - Mientras se procesa una interrupción no se pueden recibir otras interrupciones.
 - Al acabar la rutina que trataba la interrupción, se devuelve el control al proceso que se había suspendido.
- Diversas causas por las que pueden producirse interrupciones:
 - Eventos generados por el HW.
 - Interrupciones periódicas, que invocan al planificador de procesos a corto plazo del SO.
 - Llamadas al sistema, son aquellas rutinas y funciones proporcionadas por el *kernel* que las aplicaciones pueden llamar para usar de forma correcta los recursos del sistema.
 - El *kernel* proporciona una interfaz a las aplicaciones para que las tareas críticas no se puedan hacer desde la aplicación (lo que podría llevar a errores en el uso de los recursos del sistema) sino se puedan hacer solo a través del *kernel* mismo.

Kernel del sistema operativo

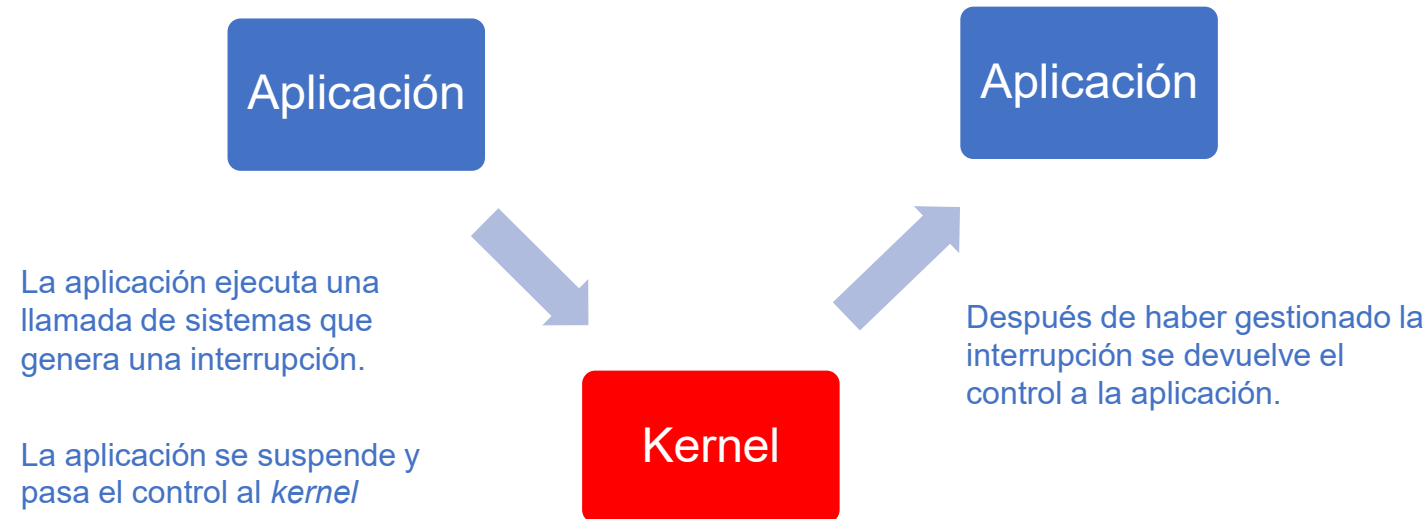
Modo usuario y modo *kernel*

El procesador normalmente funciona en modo usuario, permitiendo a las aplicaciones de ejecutarse.

Cuando el procesador tiene que ejecutar rutinas del *kernel*, pasa a modo *kernel*.

Una aplicación, cuando tiene que hacer algo critico para el sistema, usa una llamada de sistema.

La llamada de sistema generará una interrupción que suspenderá el proceso actual, ejecutará la rutina del kernel que está asociada con la interrupción y luego devolverá el control al proceso original.



Las llamadas a sistemas suelen estar escritas en un lenguaje de bajo nivel, el cual es más cercano al funcionamiento de la arquitectura del ordenador.

Estados de un proceso

Los posibles estados en los que se puede encontrar un proceso.

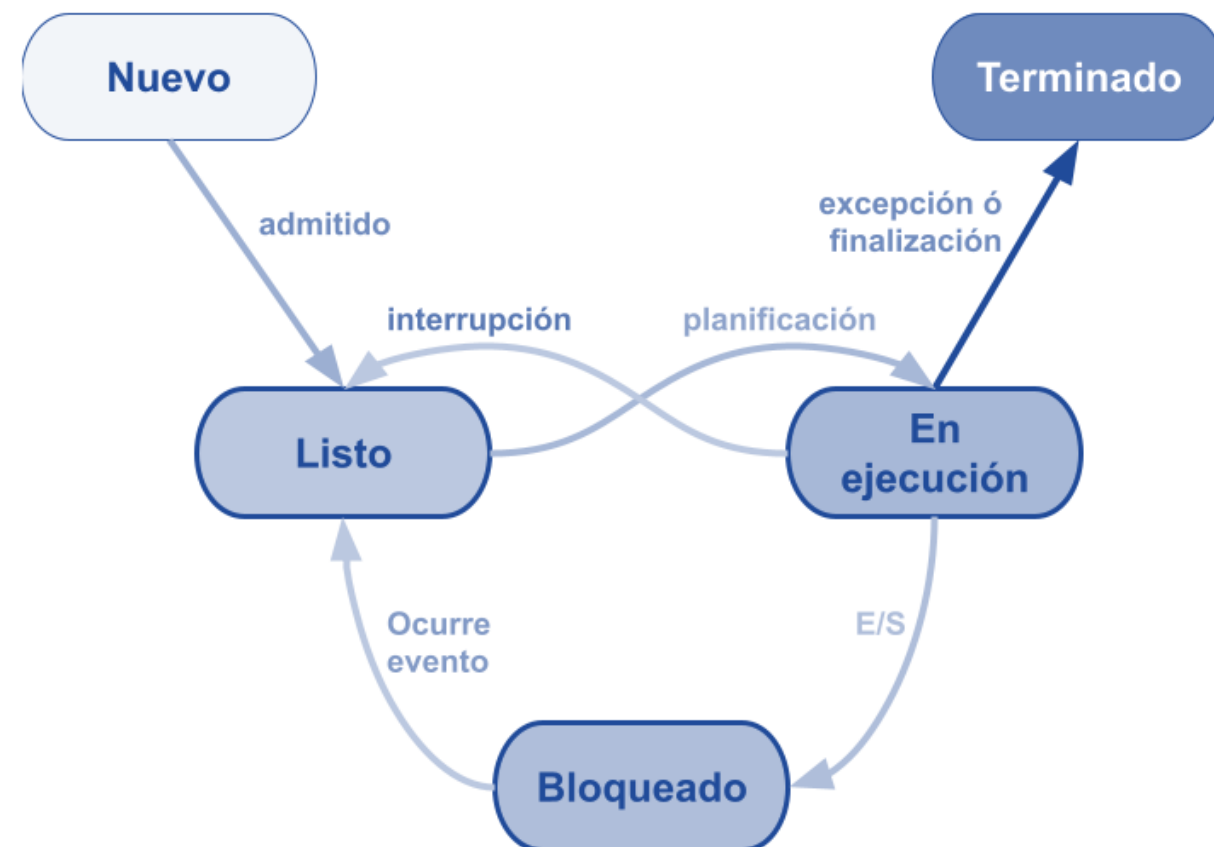
Nuevo. Cuando se carga el programa y se prepara para ser ejecutado.

Listo. Para ejecutarse cuando el SO le asigne un procesador, en función del planificador, que es el responsable de la selección del proceso que pasa al estado en ejecución.

En ejecución. Está usando el procesador. El SO utiliza el mecanismo de interrupciones para su control. Si necesita un recurso, llamará a la llamada al sistema correspondiente. Si se ejecuta durante el máximo de tiempo permitido, un temporizador lanza una interrupción.

Bloqueado. El proceso se encuentra bloqueado esperando a a que ocurra algún suceso. Por ejemplo, puede estar esperando a que termine alguna operación de E/S, o bien a sincronizarse con otro proceso.

Terminado. Al acabar, el proceso indica al sistema que ya puede liberar todos los recursos que estaba usando. Puede ser que el sistema lo termine.



Procesos en JAVA

Los procesos están representados por la clase Process.

Para crear un proceso en JAVA utilizamos la clase ProcessBuilder.

API: <https://docs.oracle.com/javase/8/docs/api/java/lang/ProcessBuilder.html>

Constructor Summary

Constructors

Constructor	Description
<code>ProcessBuilder(List<String> command)</code>	Constructs a process builder with the specified operating system program and arguments.
<code>ProcessBuilder(String... command)</code>	Constructs a process builder with the specified operating system program and arguments.

¿Cómo construir un proceso?

```
ProcessBuilder pb= new ProcessBuilder(args);
```

¿Cómo iniciar un proceso?

```
Process ps = pb.start();
```

Procesos en JAVA

¿Cómo construir un proceso? Clase `ProcessBuilder`

<https://docs.oracle.com/javase/8/docs/api/java/lang/ProcessBuilder.html>

```
ProcessBuilder pb= new ProcessBuilder(args);
```

Args puede ser:

- `String`
- `ArrayList String`
- `Vacío -> command(args);`
 - Puede recibir de atributos un `string` o `arraylist` de `string`

Arrancamos un proceso con el método `start()` que nos crea una instancia de la clase `Process` con la información del proceso lanzado.

Atributo	Descripción
<code>command</code>	La lista con el comando que deseamos lanzar y sus argumentos. Una vez creado un <code>ProcessBuilder</code> , podemos modificar esta lista mediante el atributo <code>command</code> . Para acceder a él utilizaremos el método <code>command()</code> , sin argumentos para obtener su valor y con argumentos para establecerlos.
<code>environment</code>	Contiene un mapa de las diferentes variables de entorno. Inicialmente será una copia de las variables de entorno del proceso actual. Para acceder a él utilizaremos el método <code>environment()</code> .
<code>working directory</code>	El directorio de trabajo en el que se va a ejecutar el comando. Si no se especifica, por defecto utiliza el directorio actual del proceso.

Procesos en JAVA

La clase Process

<https://docs.oracle.com/javase/8/docs/api/java/lang/Process.html>

Es una clase abstracta que encapsula la información entorno a la ejecución de un proceso.

Se puede obtener el pid del proceso mediante el método `pid()`

Método	Descripción
<code>int exitValue()</code>	Devuelve el código de salida del proceso ejecutado.
<code>Boolean isAlive()</code>	Comprueba si el proceso hijo sigue en ejecución.
<code>int waitFor()</code>	Espera que el proceso hijo finalice. El valor entero que se obtiene es el código de salida del proceso.
<code>Boolean waitFor(long timeOut, TimeUnit unidad)</code>	Se trata de una sobrecarga del método anterior, donde podemos especificar el tiempo de espera. Devolverá cierto si el proceso sigue en ejecución después del tiempo indicado y falso en caso contrario.
<code>void destroy()</code> <code>void destroyForcibly()</code>	Fuerzan la terminación del proceso, ya sea de manera controlada [SIGTERM] o forzada [SIGKILL].

Procesos en JAVA

La interfaz `ProcessHandle.Info` <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/ProcessHandle.Info.html>

Nos permite ofrecer información adicional sobre un proceso.

```
ProcessHandle.Info información = proceso.info();
```

Método	Descripción
<code>Optional <String[]> arguments()</code>	Devuelve un vector de Strings con la lista de argumentos.
<code>Optional <String> command()</code>	Devuelve un String con el path ejecutable del proceso.
<code>Optional <String> commandLine()</code>	Devuelve un String con la línea del comando introducida, concatenando el comando y los argumentos.
<code>Optional <Instant> startInstant()</code>	Devuelve la hora de inicio del proceso en un objeto de tipo Instant .
<code>Optional <Duration> totalCpuDuration()</code>	Devuelve un objeto de tipo Duration con el tiempo de CPU acumulado por el proceso.
<code>Optional <String> user()</code>	Devuelve una cadena de texto con el nombre del usuario que lanzó el proceso.

Todos los métodos devuelven un tipo `Optional`, que utiliza valores genéricos, como las colecciones. Ayuda a controlar los valores nulos en el código, para evitar los `NullPointerException`.

Podemos indicarle que nos muestre un valor concreto en caso de recibir un nulo, con el método `orElse()`.

```
System.out.println("Comando: "+info.command().orElse("-"));
```

Procesos en JAVA

La clase `Runtime`. <https://docs.oracle.com/javase/8/docs/api/java/lang/Runtime.html>

Permite acceder al entorno de ejecución de la aplicación Java.

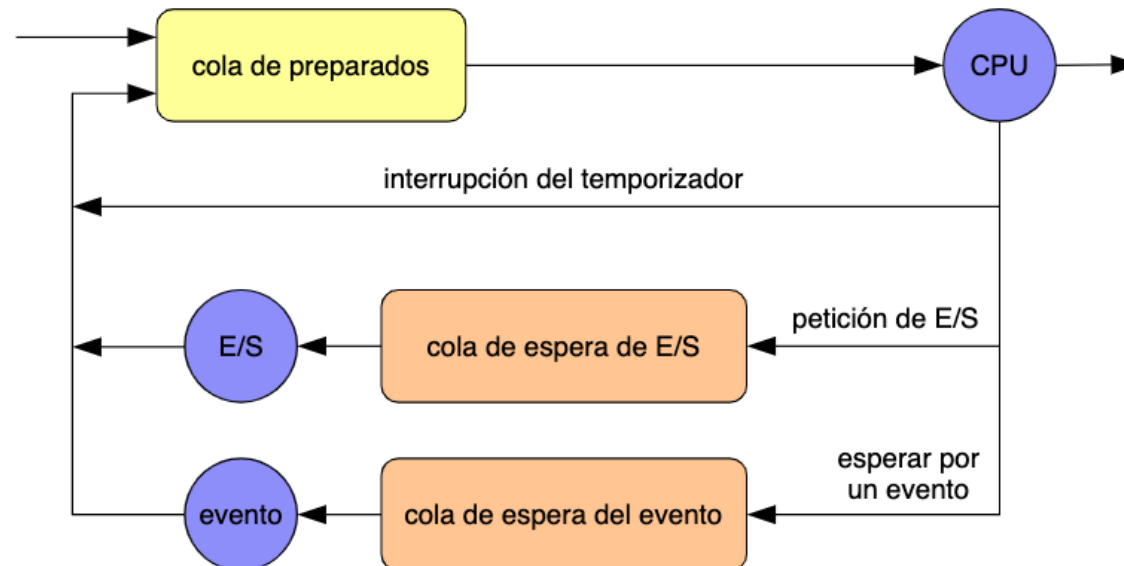
```
Runtime r = Runtime.getRuntime();  
  
Process proceso = r.exec("ping -c 3 google.com");
```

Método	Descripción
<code>int availableProcessors()</code>	Devuelve el número de procesadores disponibles para la JVM.
<code>Process exec(comando)</code>	Permite ejecutar el comando indicado. Admite diferentes firmas para indicar comando, argumentos o entorno.
<code>void exit()</code>	Finaliza la máquina virtual de Java actual.
<code>long freeMemory()</code>	Devuelve la cantidad de memoria disponible en la JVM.
<code>void gc()</code>	Permite invocar al recolector de basura de la JVM, aunque no es una buena práctica.
<code>long maxMemory()</code>	Devuelve la cantidad máxima de memoria que puede ser usada por la JVM.
<code>long totalMemory()</code>	Devuelve la cantidad total de memoria en la JVM.

Colas de procesos

Uno de los objetivos del sistema operativo es la multiprogramación, es decir, admitir varios procesos en memoria para maximizar el uso del procesador. Para ello el SO organiza los procesos en varias colas, migrándolos de unas colas a otras:

- **Cola de procesos.** Contiene todos los procesos del sistema.
- **Cola de procesos preparados.** Contiene los procesos listos en espera de ejecución.
- **Cola de procesos bloqueados.** Contiene los procesos que están a la espera de un evento o una operación de E/S.



Planificación de procesos

Se encarga de la gestión de las colas de procesos, seleccionando los movimientos de procesos entre las diferentes colas.

Existen dos tipos de planificación:

- **A corto plazo:** selecciona de la cola de procesos preparados el que pasará a ejecución. Se invoca cuando hay un cambio de estado en el proceso en ejecución, debe ser muy rápido en la decisión de elegir el siguiente proceso. Utiliza algoritmos de decisión muy sencillos:
 - Planificación sin desalojo o cooperativa. Se cambia solo si el proceso se bloquea o termina.
 - Planificación apropiativa. Como en la cooperativa, pero además si un proceso con mayor prioridad está listo, este se “apropia” del procesador.
 - Tiempo compartido. Cada cierto tiempo (cuanto), se desaloja el proceso en ejecución. Todas las prioridades de los procesos se consideran iguales.
- **A largo plazo:** selecciona qué procesos nuevos tienen que entrar en la cola de procesos preparados. (Regula el número de procesos en memoria).

Cambio de contexto

El cambio de pasar a ejecutar otro proceso conlleva que el SO guarde el contexto del proceso actual y restaure el contexto del proceso que el planificador ha elegido ejecutar.

El contexto de un proceso está formado por:

- El estado del proceso.
- El estado del procesador (registros).
- Información de gestión de memoria.

Gestión de procesos

El sistema operativo es el encargado de crear nuevos procesos, ya que es es quien gestiona los recursos.

Un proceso nuevo se crea siempre bajo petición de otro proceso: un nuevo proceso se crea porque otro proceso pide su creación en su nombre o en nombre del usuario.

Cada proceso depende de otro proceso que lo ha creado.

- Se crea un árbol de procesos.

Cada proceso tiene un identificador de proceso (PID, *process identifier*), unívoco.

Al arrancar el ordenador, se carga el *kernel* en memoria y se ejecuta un primer proceso que será raíz del árbol.

Operación con procesos

Un proceso —denominado **padre**— puede crear múltiples procesos —los **hijos**— utilizando una llamada al sistema específica para la creación de procesos, mediante una operación *create*. A su vez, los hijos pueden crear otros hijos.

Padre e hijos se ejecutan concurrentemente. Esto es, comparten la CPU. Puede que el padre tenga que esperar a resultados del hijo para poder continuar.

A la hora de que los hijos obtengan los recursos que necesitan, fundamentalmente existen dos alternativas:

- Que cada proceso hijo pueda solicitar y obtener los recursos directamente del SO, compitiendo por los recursos del sistema en las mismas condiciones que el resto de procesos en ejecución. Esta es la opción más común en los sistemas de propósito general actuales, como Microsoft Windows, Android, Linux, macOS, UNIX BSD y muchos otros.
- Que los procesos hijo solo puedan aspirar a obtener un subconjunto de los recursos de su padre. Esto es interesante en sistemas diseñados para ser muy robustos, ya que evita que un proceso pueda sobrecargar el sistema creando múltiples procesos que consuman demasiada memoria o tiempo de CPU.

Padres e hijos tienen distintos espacios de memoria asignados. Para comunicarse, pueden usar ficheros o zonas de memorias compartida.

Operación con procesos

Estas operaciones relacionadas con los procesos dependen de cada SO.

	POSIX API	Windows API
Salir	<code>exit()</code>	<code>ExitProcess()</code>
Esperar (un hijo concreto)	<code>waitpid()</code>	<code>WaitForSingleObject()</code>
Esperar (múltiples hijos)	<code>wait()</code>	<code>WaitForMultipleObject()</code>
Terminar otro proceso	<code>kill()</code>	<code>TerminateProcess()</code>

En caso de Java, la JVM proporciona una capa de abstracción que nos permite ser independientes de lo que pasa a nivel de SO.

Comunicación entre procesos

Un proceso suele tener tres canales de comunicación:

- Entrada estándar (stdin): de donde entran los datos necesarios para la ejecución del programa (no son los parámetros de ejecución, son los datos que se leen mientras se ejecuta el proceso)
- Salida estándar (stdout): donde el proceso escribe los resultados que obtiene. (System.out)
- Salida de error (stderr): donde el proceso escribe los mensajes de error. Suele ser lo mismo que el stdout, pero puede cambiar. (System.err)

Los procesos creados con `ProcessBuilder` o `Runtime.exec` no disponen una terminal asociada, por lo que no podemos trabajar directamente con las entradas y salidas del proceso.

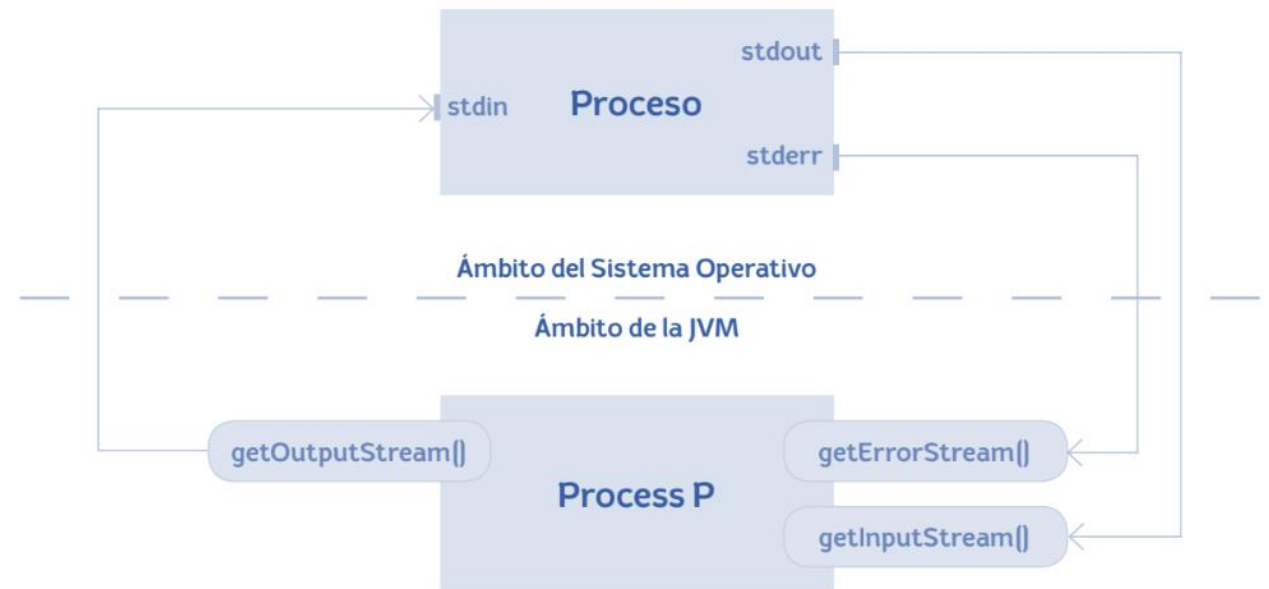
Por ello, la clase `Process` ofrece varios `Streams` o flujos de bytes asociados a la entrada y salida.

Gestión de la entrada y salida de procesos mediante Streams

Un proceso suele tener tres canales de comunicación:

- `OutputStream getOutputStream()`: devuelve un flujo de salida conectado a la entrada estándar del proceso
- `InputStream getInputStream()`: devuelve un flujo de entrada conectado a la salida estándar del proceso
- `InputStream getErrorStream()`: devuelve un flujo de salida conectado a la salida de error del proceso

Recordamos que la clase `Process` representa un proceso, pero no el proceso en sí.



Tarea 1

Realizar un programa que recibe como parámetros por línea de comandos una lista de programas ejecutables y los ejecute.

Tiene que mostrarse la información de cada proceso:

- Nombre aplicación
- PID
- Momento de inicio.

Una vez hecho, modifica el código para que, por cada proceso lanzado, espere 3 segundos y lo elimine, mostrando el código de salida del proceso.

Tarea 2

Haz un programa ProcessCom que crea otro proceso (por ejemplo, un ping) pasado por línea de comando y recupere la salida de ese proceso para escribirlo en la consola. Hay que tener en cuenta que también muestre si ha habido error.

Ten en cuenta las siguientes dos posibilidades para hacer dos versiones:

- Lectura de bytes.
- Lectura de líneas.

Redirección de la salida estándar y la salida de errores

La salida estándar como la salida de errores podemos redireccionarla a un fichero para su tratamiento posterior.

- `redirectOutput(File f)` Redirecciona el standard output hacia el fichero indicado.
- `redirectError(File f)` Redirecciona la salida de errores hacia el fichero indicado.

```
ProcessBuilder pb = new ProcessBuilder("java", "-cp", "./bin",  
"ejerciciosut1.Ej1Procesos", "calc", "explorer", "notepad");  
  
pb.redirectOutput(new File("salida.txt"));  
pb.redirectError(new File("errores.txt"));  
  
Process p = pb.start();
```

Hay que tener en cuenta que esta configuración tenemos que indicársela al programa antes de arrancar el proceso.

Modifica el programa ProcessCom para que consulte al usuario si la salida quiere que salga por consola o se guarde en un fichero salida.txt

Redirección de la salida estándar y la salida de errores

Para indicarle la redirección también podemos utilizar la clase `ProcessBuilder.Redirect`

- `ProcessBuilder.Redirect.INHERIT` Redirecciona a la salida/entrada del proceso padre.
- `ProcessBuilder.Redirect.PIPE` Se conectarán a través de una tubería el proceso Java actual y el subprocesso.
- `ProcessBuilder.Redirect.appendTo(fichero)` Para escribir al final de un fichero.
- `ProcessBuilder.Redirect.To(fichero)` Para escribir en un fichero

```
ProcessBuilder pb = new ProcessBuilder("java", "-cp", "./bin", "ejerciciosut1.Ej1Procesos", "calc",  
"explorer", "notepad");  
pb.redirectOutput(ProcessBuilder.Redirect.INHERIT);  
Process p = pb.start();
```

```
ProcessBuilder pb = new ProcessBuilder("java", "-cp", "./bin", "ejerciciosut1.Ej1Procesos", "calc",  
"explorer", "notepad");  
pb.redirectOutput(ProcessBuilder.Redirect.appendTo(new File("salida.txt")));  
Process p = pb.start();
```

```
ProcessBuilder pb = new ProcessBuilder("java", "-cp", "./bin", "ejerciciosut1.Ej1Procesos", "calc",  
"explorer", "notepad");  
pb.redirectOutput(ProcessBuilder.Redirect.To(new File("salida2.txt")));  
Process p = pb.start();
```

Otras formas de comunicación

Los procesos pueden comunicarse usando otros mecanismos:

- **Sockets:** lo veremos en futuro
- **JNI (Java Native Interface):** Java normalmente abstrae del SO subyacente. En determinados casos podríamos necesitar funcionalidades a bajo nivel que nos proporcionaría el uso de otro lenguaje como C. JNI permite la llamada desde Java a aplicaciones desarrolladas en otros lenguajes.
- **Librerías de comunicación no estándares:** hay librerías que incluyen código nativo que permite trabajar con conceptos de más bajo nivel (memoria compartida, pipes, semáforos).

Sincronización de procesos

Los métodos de comunicación se pueden considerar métodos de sincronización.

- Las comunicaciones entre padre e hijo usan un canal de comunicación unidireccional bloqueante
- Si el padre pide leer la salida stdout del hijo usando un `InputStream` se bloquea hasta que el hijo envía datos.

Lanzar un programa desde otro programa

Para lanzar un programa Java desde otro programa tenemos que usar el comando `java` seguido del nombre `paquete.programa.class` (pero sin extensión). Tenemos que indicarle el directorio en el que se encuentra.

```
java -cp ./bin paquete.programa [args]
```

Tarea 4.

Crea un programa RunAndWait que lanza otro proceso Sumador que se pasa por línea de comando con sus argumentos. El resultado de la suma tiene que guardarse en un fichero .txt con nombre que le pasemos por parámetro.

Haz que el proceso padre espere al finalizar del proceso hijo y luego escriba en pantalla qué comando se ha lanzado y qué devolvió.

Modifica el programa anterior (llámalo ModRunAndWait) para que lance tres procesos Sumador que se pasan por línea de comando con sus argumentos. El resultado de la suma tiene que guardarse en un fichero .txt cuyo nombre que le pasemos también por parámetro.

Haz que el proceso padre deberá acceder a todos los ficheros con el resultado y hacer la suma total y mostrarla por pantalla.

Programación Multiproceso

Para crear un programa multiproceso cooperativo se tienen que tener en cuenta distintos aspectos:

1. **Descomposición funcional:** identificar las funciones de la aplicación y qué relaciones tienen entre ellas
2. **Partición:** distribuir las funciones entre los procesos y establecer un esquema de comunicación entre ellos. (maximizar la independencia, puesto que sincronizar procesos cuesta tiempo)
3. **Implementación:** implementar la solución pensada.

Práctica UT1

Crea un programa SumWorker que lee un número desde su entrada estándar, le suma uno y lo escribe en su salida estándar.

El proceso SumWorker no termina nunca. Una vez hecho su trabajo vuelve a hacerlo otra vez.

Crea un programa SumManager, que lanza un proceso SumWorker y luego pide un número al usuario. El número recibido se envía al SumWorker y la salida se lee en la consola de SumManager. Si el usuario escribe un número negativo, el programa termina.

Gracias
